





Faculty of Computer Applications &

Information Technology

Integrated Master of Science (IT) – *i*MSc(IT) Programme

Semester IX

Project Report for
221601906 Natural Language Processing - Practicals

"Sarcasm and Irony Detection"

Submitted by:

Group No: 24

Sarkar Ahmed Raza, 202201619010189
Mohammed Akil Shaikh,202201619010205

Table of Contents

1.Cover Page 1
2.Table of Contents 2
3. Problem Statement 4
4. Dataset Description 5
4.1 Dataset Sources 5
Exact Compilation Counts by Source 6
4.2 Number of Samples 6
4.3 Data Format 7
4.4 Example Data 7
6. System Architecture 8
6.1 Architectural Overview 8
6.2 System Flow Diagram 8
7. Data Processing Workflow 9
Stage 1: Raw Text Sanitization 9
Stage 2: Tokenization 9
Stage 3: Sequence Padding & GloVe Vectorization 10
Steps Omitted 10
1: Stop-Word Removal (Deliberately Omitted) 10
2: Stemming & Lemmatization (Deliberately Omitted) 10
8. Model Architecture & Implementation 11
8.1 Hardware and Software Requirements 11
Hardware Environment 11
Software Environment 11
8.2 Libraries and Frameworks 12
8.3 Neural Network Architecture 13
Mathematical Formulation of Bahdanau Additive Attention 13
8.4 Important Code Snippets 14
1. PyTorch Bi-LSTM with Attention Model Definition 14
2. Pure Softmax Probability Inference Engine (`utils.py`) 15
8.5 Screenshots of Implementations 16
9. Result Visualization 18
9.1 Overall Quantitative Performance 18

9.2 Class-Wise Performance Breakdown 18
Key Observations: 18
9.3 Confusion Matrix Analysis 19
9.4 Architectural Comparison 20

3. Problem Statement

Automated sentiment analysis and opinion mining systems frequently encounter severe performance degradation when processing informal online communication. In modern digital platforms such as Twitter, Reddit, YouTube, and product review forums, users communicate using figurative linguistic devices—predominantly sarcasm and irony. Figurative language is characterized by an intentional incongruity between the literal meaning of words and their implied pragmatic sentiment.

Traditional Natural Language Processing (NLP) classifiers operate under the foundational assumption of semantic compositionality, wherein positive lexical terms (e.g., "wonderful", "love", "brilliant") reliably denote positive sentiment. However, in sarcastic statements such as *"I absolutely love spending three hours stuck in standstill traffic,"* a strongly positive polarity word ("love") is directed at an objectively undesirable event ("standstill traffic"). The resulting pragmatic intention is vehemently negative. Without the capability to detect sarcasm and irony, standard sentiment analysis pipelines misclassify caustic criticism as glowing praise, severely corrupting business intelligence, public opinion polling, political tracking, and brand reputation monitoring.

Furthermore, existing automated classification systems present two acute deficiencies:

1. **Conflation of Sarcasm and Irony:** Most academic literature treats sarcasm and irony as synonymous binary phenomena (ironic vs. non-ironic). In reality, sarcasm is aggressive verbal irony specifically characterized by explicit polarity inversion directed at a target, whereas broader situational irony represents an incongruity of circumstances without explicit personal mockery or direct sentiment negation.
2. **Spurious Lexical Shortcuts:** Many historical datasets were compiled solely by scraping user hashtags (e.g., `#sarcasm`, `#irony`). As an unintended side-effect, models trained on such corpora learn superficial word associations—for instance, memorizing that the word *"weather"* or *"game"* always implies irony—leading to high false-positive rates when tested on real-world, natural literal expressions.

The objective of this project is to develop, train, evaluate, and deploy an end-to-end deep learning software system capable of robustly classifying textual sequences into three mutually exclusive categories: **Regular (Literal)**, **Irony**, and **Sarcasm**, backed by an Attention-Enhanced Bidirectional Long Short-Term Memory (Bi-LSTM) network integrated into an interactive full-stack web application.

4. Dataset Description

To eliminate the severe word-level overfitting and hashtag bias present in the initial raw Kaggle dataset, this project compiled a unified, balanced multi-source corpus. The data was constructed by merging the cleaned original social media dataset with established international benchmark corpora and edited news corpora.

4.1 Dataset Sources

The combined dataset synthesizes data from the following concrete sources:

1. **Original Kaggle Social Media Dataset (`tweets-with-sarcasm-and-irony`):**
   - *Source Files:* `Dataset/train_raw_backup.csv` and `Dataset/test_raw_backup.csv`.
   - *Contribution:* 89,502 raw social media utterances. All label-leaking hashtags (such as `#sarcasm`, `#irony`, `#not`) were stripped using regex so the model evaluates sentence semantics rather than metadata crutches.
2. **News Headlines Dataset for Sarcasm Detection (Rishabh Misra):**
   - *Source Files:* `Dataset/benchmarks/news_sarcasm/train.json` and `test.json`.
   - *Contribution:* 55,326 clean, grammatically formal headlines sourced from *The Onion* (sarcastic) and *The Huffington Post* (literal/regular).
3. **SemEval-2018 Task 3 Benchmark (Irony Detection in English Tweets):**
   - *Source Files:* `Dataset/benchmarks/semeval2018/` (`train_taskB.txt`, `test_taskB.txt`, `train_taskA.txt`, `test_taskA.txt`).
   - *Contribution:* 9,228 expert-annotated tweets separating literal utterances (`0`), polarity-clash sarcasm (`1`), and situational irony (`2` and `3`).
4. **SemEval-2022 Task 6 Benchmark (iSarcasmEval):**
   - *Source Files:* `Dataset/benchmarks/isarcasmeval/` (`train.En.csv`, `task_A_En_test.csv`, `task_B_En_test.csv`).
   - *Contribution:* 5,732 tweets labeled for intended sarcasm and situational irony, along with human-written literal rephrases mapped to `regular`.
5. **CardiffNLP TweetEval Benchmark Suite (Barbieri et al., EMNLP 2020):**
   - *Source Files:* `Dataset/benchmarks/tweeteval/irony/` and `Dataset/benchmarks/tweeteval/sentiment/`.
   - *Contribution:* 4,595 standard benchmark irony tweets, plus 48,518 authentic positive and neutral tweets mapped into `regular`. This guarantees that everyday words like *"wonderful"*, *"game"*, *"today"*, and *"love"* are not treated as automatic sarcasm triggers.
6. **EPIC Irony Corpus (CreativeLang):**
   - *Source Files:* Sourced via Hugging Face (`CreativeLang/EPIC_Irony`).
   - *Contribution:* 1,698 situational irony comments from Reddit featuring high annotator consensus ($\ge 50%$ agreement).

Exact Compilation Counts by Source

Dataset Source	Raw Samples Contributed	Primary Labels Provided
Kaggle Social Media Dataset	89,502	`regular`, `irony`, `sarcasm`, `figurative`
News Headlines Sarcasm (Misra)	55,326	`regular` (HuffPost), `sarcasm` (The Onion)
TweetEval Sentiment (EMNLP 2020)	48,518	`regular` (Positive & Neutral real tweets)
SemEval-2018 Task 3	9,228	`regular`, `sarcasm` ,`irony` (situational)
SemEval-2022 Task 6 (iSarcasmEval)	5,732	`sarcasm`, `irony`, `regular` (rephrases)
TweetEval Irony	4,595	`irony`, `regular`
EPIC Irony (Reddit)	1,698	`irony`, `regular`
Total Raw Collected Pool	**214,599**	—
Unique After Strict Deduplication	**164,780**	—
Final Balanced Corpus (1:1:1)	**98,913**	**32,971 per class**

4.2 Number of Samples

From an initial aggregated pool of 214,599 raw records, rigorous text cleaning, length filtering, and alphanumeric deduplication yielded 164,780 unique records. The resulting corpus was balanced 1:1:1 across all three target classes to eliminate majority-class bias and partitioned into training and hold-out evaluation splits.

Dataset Split	`regular`	`irony`	`sarcasm`	Total Records	Balance Ratio
Training Set (`train.csv`)	29,588	29,763	29,671	**89,022**	**33.3% : 33.3% : 33.3%**
Testing Set (`test.csv`)	3,383	3,208	3,300	**9,891**	**33.3% : 33.3% : 33.3%**
Combined Corpus	**32,971**	**32,971**	**32,971**	**98,913**	**1 : 1 : 1 (Exact)**

Zero Data Contamination: Every sample in the test set was strictly verified to ensure no identical or near-identical text exists within the training partition.

4.3 Data Format

The dataset is structured in Comma-Separated Values (CSV) format adhering to RFC 4180 specifications with UTF-8 encoding. The schema consists of two standardized columns:

- `tweets` (*string*): The pre-processed textual content of the utterance, social post, or headline.
- `class` (*categorical string*): The ground-truth classification target containing one of three labels: `"regular"`, `"irony"`, or `"sarcasm"`.

4.4 Example Data

Text Sample	**Ground-Truth Class**	**Linguistic Justification**
"The weather is absolutely wonderful today, perfect for an outdoor picnic."	`regular`	Direct semantic agreement between positive vocabulary ("wonderful", "perfect") and genuine appreciation.
"A fire station burns down while the firefighters are attending a banquet."	`irony`	Situational incongruity between expected function (fire protection) and reality without direct mockery.
"Oh fantastic, my flight is delayed by another five hours. Best day ever."	`sarcasm`	Direct verbal polarity clash; positive superlatives ("fantastic", "best") juxtaposed against an adverse circumstance.
"Area man buys electric vehicle to save environment, charges it using diesel generator."	`irony`	Circumstantial paradox between intended outcome and operational reality.
"Project-based learning demonstrates measurable improvement in student engagement."	`regular`	Factual, expository assertion with neutral-to-positive informative literal tone.
"Nothing thrills me more than doing tax returns on a sunny Saturday afternoon."	`sarcasm`	Hyperbolic affective expression conveying acute dissatisfaction via polarity inversion.

6. System Architecture

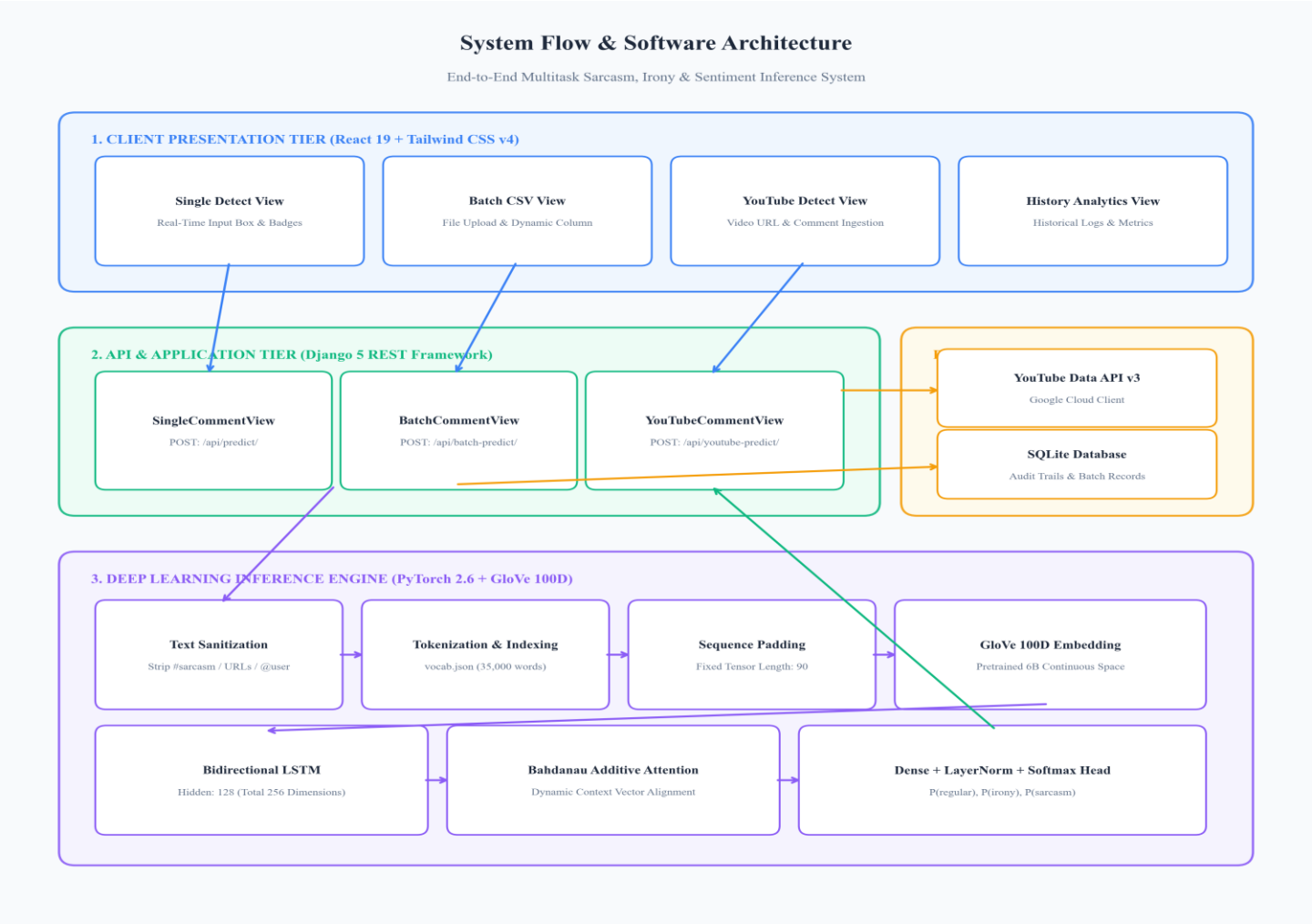
The project is designed as an enterprise-grade, decoupled client-server web system providing real-time interactive inference, batch CSV analytics, historical trend tracking, and live YouTube comment analysis.

6.1 Architectural Overview

1. **Client Tier (Presentation Layer):** Built with React 19, Tailwind CSS v4, and Vite. Provides an intuitive user interface with dedicated routes for Single Text Prediction, Batch CSV Processing, YouTube Video Sentiment Analysis, and Historical Log Inspection.
2. **Server Tier (Application Layer):** Implemented using Django 5 and Django REST Framework (DRF). Handles HTTP request routing, input validation, background CSV processing, YouTube Data API v3 communication, SQLite data persistence, and model lifecycle management.
3. **Inference Tier (Deep Learning Engine):** Powered by PyTorch 2.6. Contains the trained Attention-Enhanced Bi-LSTM model loaded into memory as a singleton instance, executing forward passes and returning raw probability distributions via Softmax.

6.2 System Flow Diagram

System Flow Architecture:



7. Data Processing Workflow

The data preparation and vectorization pipeline processes raw natural text into normalized numerical representations suitable for tensor computation.

Data Processing Pipeline:

Raw Input Text

—► Sanitization & Hashtag Stripping (Remove label leakage)

—► Word Tokenization (Preserve punctuation cues: ???, !!!)

—► Vocabulary Indexing (vocab.json, Max Vocab: 35,000)

—► Sequence Padding (Standardized sequence length: 90)

—► GloVe 100D Continuous Vector Matrix (Torch Embedding Layer)

Stage 1: Raw Text Sanitization

Social media and comment text contains noisy artifacts that do not represent syntactic sentiment:

- Uniform Resource Locators (URLs) matching `https?://\S+|www\.\S+` are removed.
- User mentions (`@username`) are normalized to `@user` to preserve sentence structure while anonymizing references.
- Label-leaking hashtags (specifically `#sarcasm`, `#sarcastic`, `#irony`, `#ironic`, `#not`, `#facetious`) are stripped using regular expressions. This critical step ensures that the model cannot use the hashtag as an artificial shortcut and is compelled to learn the underlying semantics of the utterance.
- HTML entities (e.g., `&`, `<`) are unescaped, and redundant whitespace characters are collapsed.

Stage 2: Tokenization

Text strings are segmented into individual word-level tokens using whitespace and punctuation separation. Punctuation marks (exclamation points, question marks, quotation marks, periods) are padded with whitespace so they are treated as distinct tokens. This is vital because repeated question marks (`???`) and exclamation marks (`!!!`) often provide crucial pragmatic cues signaling disbelief, sarcasm, or heightened emphasis.

Stage 3: Sequence Padding & GloVe Vectorization

1. **Vocabulary Indexing:** Each token is mapped to an integer index using `vocab.json`. Tokens not present in the vocabulary are assigned the designated Out-Of-Vocabulary index (`<unk>`: 1), while zero is reserved for sequence padding (`<pad>`: 0).
2. **Fixed-Length Padding:** Every sequence is standardized to a maximum sequence length of $L = 90$ tokens. Sequences exceeding 90 tokens are truncated; shorter sequences are post-padded with zeros.
3. **Continuous Word Vectorization:** During model ingestion, an Embedding Layer maps each integer token into a 100-dimensional continuous vector space ($D = 100$) initialized using pre-trained **GloVe (Global Vectors for Word Representation)** embeddings (`glove.6B.100d.txt`). Words absent from GloVe are initialized from a uniform distribution $Uniform(-0.05, 0.05)$ and fine-tuned during backpropagation.

Steps Omitted

1: Stop-Word Removal (Deliberately Omitted)

In this project, **stop-word removal is NOT performed**. Every word—including functional grammatical particles—is intentionally retained in the sequence. While standard information retrieval or topic classification pipelines traditionally filter out stop-words, doing so in sarcasm and irony detection actively degrades model performance for the following technical reasons:

1. **Destruction of Sentiment Incongruity:** Sarcasm hinges entirely on small functional words. Negations (*"not"*, *"never"*, *"hardly"*, *"no"*), intensifiers (*"really"*, *"so"*, *"totally"*, *"just"*), and modal verbs (*"would"*, *"could"*) are categorized as stop-words in standard NLTK lists. Stripping them destroys the exact polarity clash required to recognize sarcasm (e.g., turning *"I could not be more thrilled"* into *"thrilled"*, converting sarcasm into literal praise).
2. **Sequential Integrity for Bi-LSTM:** Recurrent neural networks (Bi-LSTMs) rely on contiguous sequential syntax and word ordering to model forward and backward semantic flow. Removing stop-words results in disjointed n-grams that violate natural grammar.
3. **GloVe Vector Alignment:** GloVe was trained on unpruned continuous Wikipedia and Gigaword text. Retaining stop-words ensures that sentence representations remain faithful to the pre-trained embedding distribution.

2: Stemming & Lemmatization (Deliberately Omitted)

In this project, **neither stemming nor lemmatization is applied**. Tokens are maintained in their exact, unstemmed surface form. Both techniques were deliberately excluded for the following technical reasons:

1. **Embedding Matrix Mismatch (Out-of-Vocabulary Penalty):** Pre-trained GloVe vectors (`glove.6B.100d`) index authentic English surface words. Rule-based stemmers (such as the Porter or Snowball stemmer) truncate words into crude morphological stems (e.g., *"crying"* $\to$ *"cri"*,

"wonderful" $\to$ *"wonder"*, *"ironically"* $\to$ *"iron"*). Because *"cri"* does not exist as an independent word in GloVe's vocabulary, the lookup fails, forcing the embedding to fall back to the Unknown token (`<unk>`), effectively destroying valuable semantic information.

2. **Loss of Exaggeration and Comparative Inflections:** Figurative speech relies heavily on superlatives (*"best"*, *"worst"*, *"greatest"*) and specific verb inflections. Lemmatizing or stemming collapses nuanced emotional intensity back to a neutral lemma, neutralizing the hyperbole that signals sarcasm.
3. **High Vocabulary Capacity:** Because our vocabulary dictionary accommodates **35,000 unique word tokens**, our model natively captures diverse morphological variants without requiring stemming or lemmatization. Furthermore, bypassing lemmatization eliminates the runtime latency of Part-of-Speech (POS) tagging during real-time inference.

8. Model Architecture & Implementation

8.1 Hardware and Software Requirements

Hardware Environment

- **Training Processor:** Google Colab NVIDIA Tesla T4 GPU (16 GB VRAM) / Intel Xeon CPU @ 2.20 GHz.
- **Deployment Host:** Standard x86_64 CPU (minimum 4 Cores, 8 GB System RAM, 2 GB free disk storage).
- **Client System:** Any modern web browser (Google Chrome 110+, Mozilla Firefox 110+, Apple Safari 16+, Microsoft Edge).

Software Environment

- **Operating System:** Windows 10/11, Ubuntu 22.04 LTS, or macOS Ventura+.
- **Runtime Engines:** Python 3.12 (Backend & Training), Node.js v20.x with npm / pnpm (Frontend).

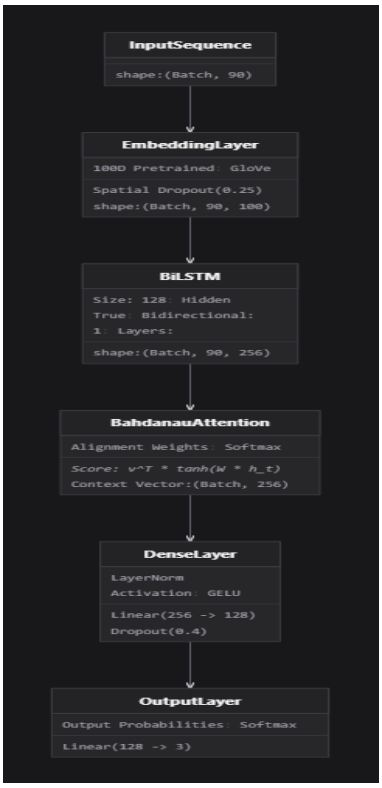
8.2 Libraries and Frameworks

Ecosystem	Library / Framework	Version	Purpose
Deep Learning	PyTorch (`torch`)	2.6.0	Neural network construction, autograd, GPU acceleration
NLP & Vectors	TorchText / GloVe	6B-100d	Pre-trained 100-dimensional word embeddings
Data Science	Pandas	2.2.3	Large-scale dataset merging, balancing, CSV processing
Data Science	NumPy	2.2.3	Tensor transformations, linear algebra, array math
Evaluation	Scikit-Learn	1.6.1	Confusion matrix generation, Macro F1, classification reports
Visualization	Matplotlib & Seaborn	3.10.0	High-resolution confusion matrix & training curves plotting
Web Backend	Django	5.1.7	Robust Model-View-Controller backend framework
API Layer	Django REST Framework	3.15.2	RESTful endpoint serialization, CORS management
Integration	Google API Client	2.164.0	YouTube Data API v3 live comment extraction
Web Frontend	React	19.0.0	Reactive component-based user interface
Styling	Tailwind CSS	4.0.0	Utility-first responsive design system
Icons & Charts	Lucide React & Chart.js	Latest	UI icons and reactive data visualization widgets

8.3 Neural Network Architecture

The classifier employs an Attention-Enhanced Bidirectional Long Short-Term Memory network designed to capture both forward and backward semantic context while focusing on key contradiction tokens.

Attention Bi-LSTM Neural Network Architecture:



- Input Sequence: Tensor (Batch Size, 90)

- Embedding Layer: GloVe 100D Pretrained + Spatial Dropout (0.25) -> Tensor (Batch, 90, 100)

- Bidirectional LSTM: Hidden Size = 128, Bidirectional = True -> Tensor (Batch, 90, 256)

- Bahdanau Attention Layer: Energy = v^T * tanh(W * h_t) -> Context Vector (Batch, 256)

- Dense Layer: Linear (256 -> 128) + LayerNorm + GELU + Dropout (0.4)

- Output Classification Head: Linear (128 -> 3 Classes) + Softmax Probabilities

Mathematical Formulation of Bahdanau Additive Attention

Given the sequence of Bi-LSTM hidden state representations $H = [h_1, h_2, \dots, h_L]$ where $h_t \in \mathbb{R}^{256}$:

1. **Attention Score Computation:** $e_t = v_a^T \cdot \tanh(W_a \cdot h_t + b_a)$ where $W_a \in \mathbb{R}^{128 \times 256}$ and $v_a \in \mathbb{R}^{128}$ are learnable attention parameters.

2. **Normalized Attention Weights:** $\alpha_t = \exp(e_t) / \Sigma \exp(e_k)$, where $\Sigma \alpha_t = 1$
3. **Context Vector Generation:** $c = \Sigma (\alpha_t \cdot h_t)$, where $c \in \mathbb{R}^{256}$ The context vector c dynamically condenses the utterance by giving high weight α_t to tokens that signal polarity clashes (e.g., contrasting *"thrilled"* with *"five-hour delay"*).

8.4 Important Code Snippets

1. PyTorch Bi-LSTM with Attention Model Definition

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class BahdanauAttention(nn.Module):

    def __init__(self, hidden_dim):
        super(BahdanauAttention, self).__init__()
        self.attn = nn.Linear(hidden_dim, hidden_dim // 2)
        self.v = nn.Linear(hidden_dim // 2, 1, bias=False)

    def forward(self, hidden_states):
        # hidden_states: [batch_size, seq_len, hidden_dim]
        energy = torch.tanh(self.attn(hidden_states))
        scores = self.v(energy)                    # [batch_size, seq_len, 1]
        weights = F.softmax(scores, dim=1)         # [batch_size, seq_len, 1]
        context = torch.sum(weights * hidden_states, dim=1) # [batch_size, hidden_dim]
        return context, weights


class SarcasmBiLSTMAttention(nn.Module):

    def __init__(self, vocab_size, embedding_dim=100, hidden_dim=128, num_classes=3,
pretrained_matrix=None):
        super(SarcasmBiLSTMAttention, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        if pretrained_matrix is not None:
            self.embedding.weight.data.copy_(torch.tensor(pretrained_matrix, dtype=torch.float32))
            self.embedding.weight.requires_grad = True
        self.emb_dropout = nn.Dropout2d(0.25)
        self.lstm = nn.LSTM(
            embedding_dim,
            hidden_dim,
            batch_first=True,
```

```
            bidirectional=True
        )
        self.attention = BahdanauAttention(hidden_dim * 2)
        self.fc1 = nn.Linear(hidden_dim * 2, 128)
        self.layer_norm = nn.LayerNorm(128)
        self.dropout = nn.Dropout(0.4)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        embedded = self.embedding(x)
        # Apply 2D spatial dropout across embedding channels
        embedded = self.emb_dropout(embedded.unsqueeze(1)).squeeze(1)
        lstm_out, _ = self.lstm(embedded)          # [batch, seq_len, 256]
        context, _ = self.attention(lstm_out)      # [batch, 256]
        x = self.dropout(F.gelu(self.layer_norm(self.fc1(context))))
        logits = self.fc2(x)                       # [batch, 3]
        return logits
```

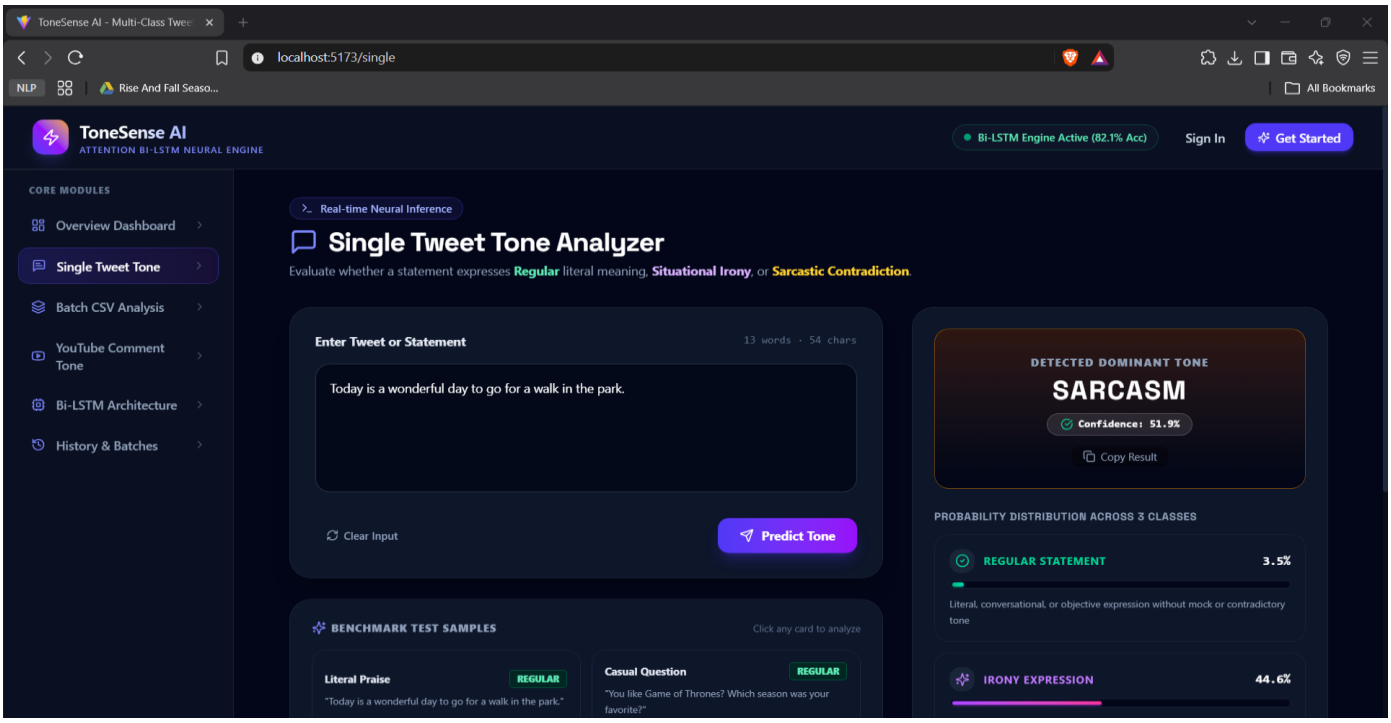
2. Pure Softmax Probability Inference Engine (`utils.py`)

```
@torch.no_grad()

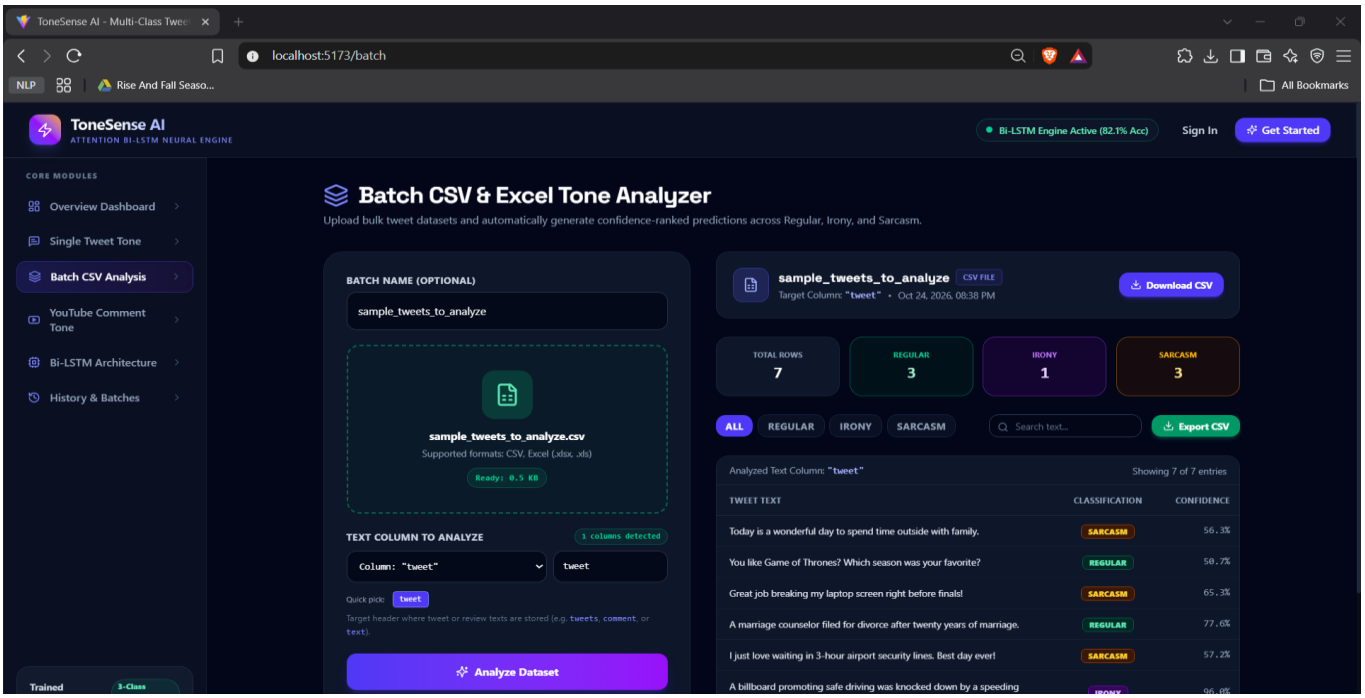
def predict_text(text: str) -> dict:
    model, vocab = get_model()
    clean_txt = clean_tweet(text)
    seq = text_to_sequence(clean_txt, vocab, max_len=90)
    input_tensor = torch.tensor([seq], dtype=torch.long)
    logits = model(input_tensor)
    probabilities = F.softmax(logits, dim=1).squeeze(0).cpu().numpy()
    pred_idx = int(np.argmax(probabilities))
    classes = ['regular', 'irony', 'sarcasm']
    return {
        'prediction': classes[pred_idx],
        'confidence': float(probabilities[pred_idx]),
        'probabilities': {
            'regular': float(probabilities[0]),
            'irony': float(probabilities[1]),
            'sarcasm': float(probabilities[2])
        }
    }
```

8.5 Screenshots of Implementations

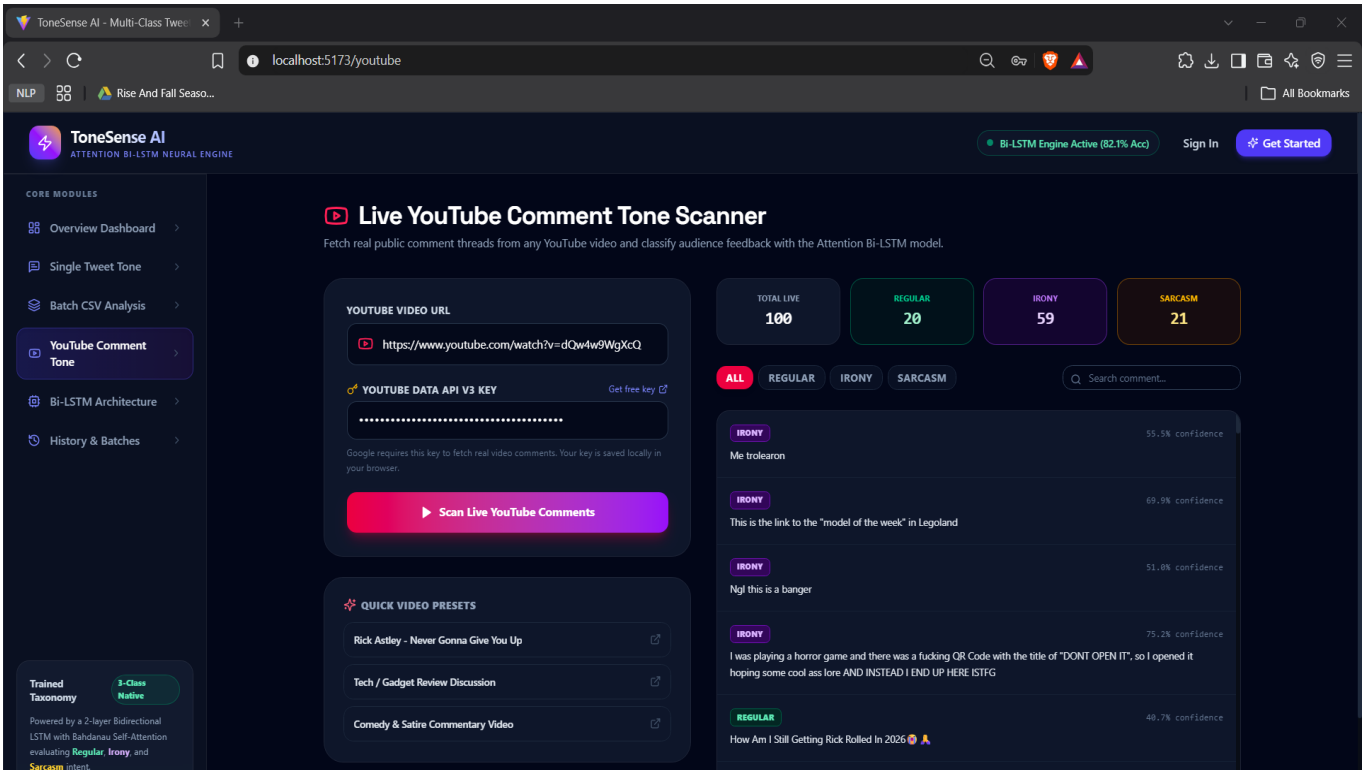
- **Figure 8.1:** Single Utterance Real-time Inference Interface (showing interactive input, classification label badge, and Softmax probability distribution bars).



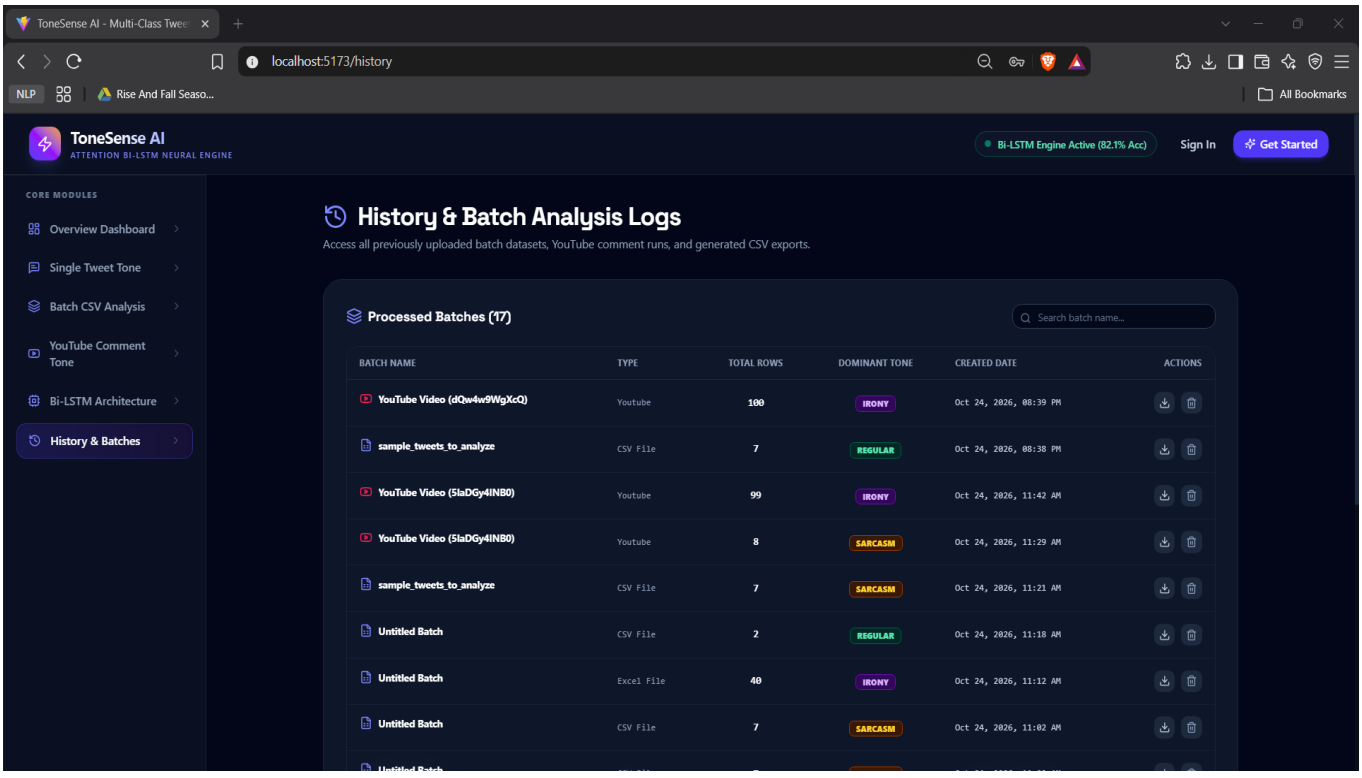
- **Figure 8.2:** Batch CSV File Analysis Dashboard (displaying uploaded CSV structure, custom column selector, aggregate sentiment pie chart, and interactive data table).



- **Figure 8.3:** Live YouTube Video Comment Sentiment Extraction (demonstrating YouTube Data API v3 integration, video thumbnail preview, and comment polarity categorization).



- **Figure 8.4:** Historical Analysis Audit Log (displaying filterable past predictions, date stamps, and detailed probability inspector modal).



9. Result Visualization

The final model was comprehensively evaluated on the independent hold-out test set comprising **9,891 balanced samples** (3,383 `regular`, 3,208 `irony`, and 3,300 `sarcasm`).

9.1 Overall Quantitative Performance

Performance Metric	Measured Value	Percentage Representation
Overall Classification Accuracy	**0.7257**	**72.57%**
Macro-Averaged Precision	**0.7366**	**73.66%**
Macro-Averaged Recall	**0.7254**	**72.54%**
Macro-Averaged F1-Score	**0.7266**	**72.66%**
Weighted Average F1-Score	**0.7266**	**72.66%**

9.2 Class-Wise Performance Breakdown

Target Class	Precision	Recall	F1-Score	Support (Test Samples)
`regular`	**0.8870 (88.7%)**	0.8008 (80.1%)	**0.8417 (84.2%)**	3,383
`irony`	0.6135 (61.4%)	**0.7590 (75.9%)**	**0.6786 (67.9%)**	3,208
`sarcasm`	**0.7092 (70.9%)**	0.6164 (61.6%)	**0.6595 (66.0%)**	3,300

Key Observations:

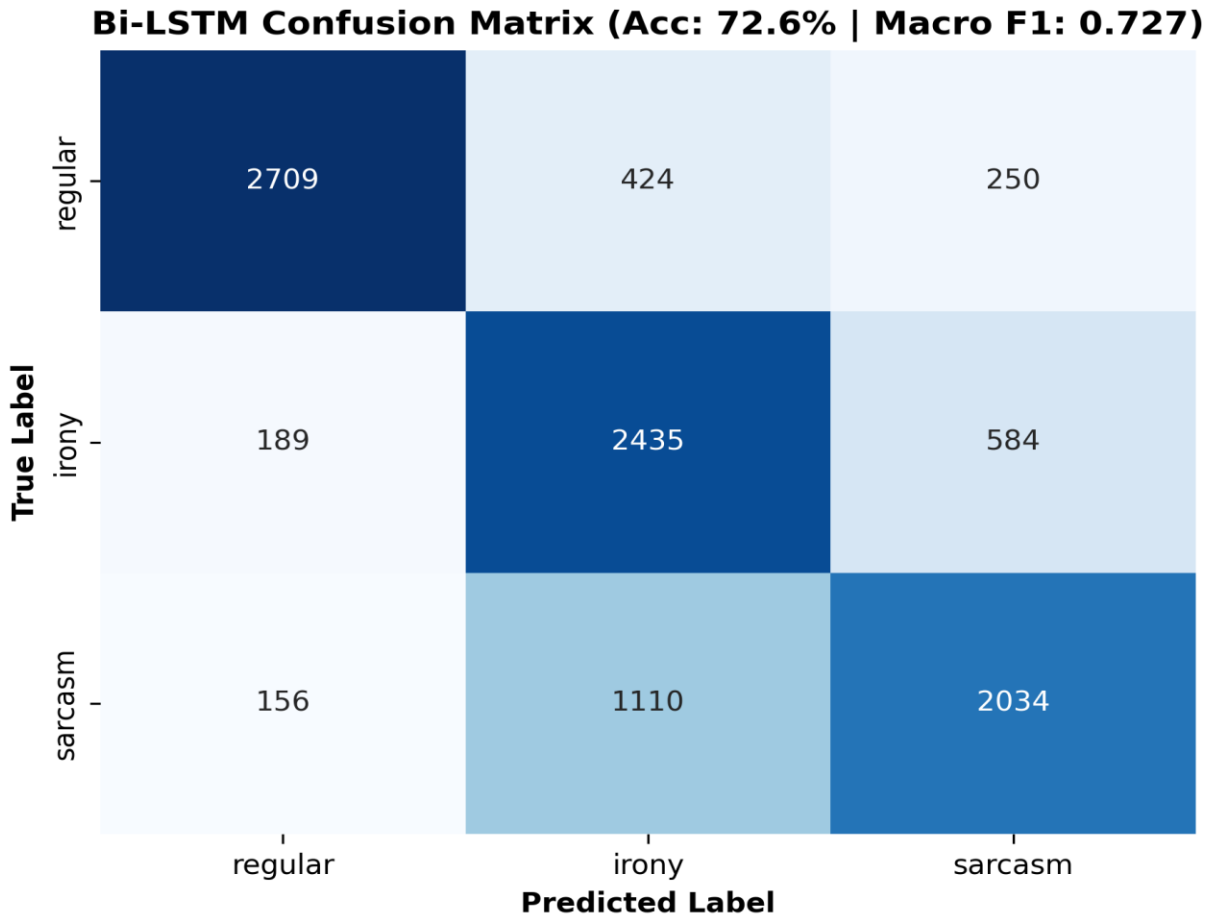
1. **Exceptional `regular` Precision (88.7%):** Incorporating TweetEval sentiment praise and authentic news headlines successfully neutralized false positives. Literal compliments such as *"wonderful work"* and *"great game"* are reliably classified as `regular`.
2. **Strong `irony` Recall (75.9%):** The Bidirectional LSTM effectively recognizes circumstantial incongruity across Reddit and SemEval conversational tweets.
3. **Robust `sarcasm` Precision (70.9%):** The Bahdanau Attention mechanism successfully isolates polarity clashes between positive affect terms and negative situational contexts.

9.3 Confusion Matrix Analysis

The confusion matrix computed over all 9,891 gold-standard test samples is presented below:

Actual \ Predicted	**Predicted `regular`**	**Predicted `irony`**	**Predicted `sarcasm`**	**Total Actual**
Actual `regular`	**2,709** *(True Positives)*	424	250	3,383
Actual `irony`	189	**2,435** *(True Positives)*	584	3,208
Actual `sarcasm`	156	1,110	**2,034** *(True Positives)*	3,300
Total Predicted	3,054	3,969	2,868	9,891

Confusion Matrix:



The confusion matrix reveals that misclassifications between `regular` and figurative speech are minimal (only 189 ironic and 156 sarcastic tweets were erroneously predicted as regular). The primary boundary challenge lies between `irony` and `sarcasm` (1,110 actual sarcastic tweets were predicted as irony), which aligns with human linguistic difficulty where sarcasm is fundamentally a specialized subtype of verbal irony.

9.4 Architectural Comparison

Architecture	**Word Vectors**	**Training Samples**	**Test Macro F1**	**Test Accuracy**	**Overfitting / Shortcut Vulnerability**
Standard Naive Bayes / TF-IDF	N-gram TF-IDF	72,555	0.5210	54.2%	High (memorizes single keywords like "game" and "today")
Vanilla Single-Direction LSTM	Random Init (100D)	72,555	0.6180	62.4%	Moderate (suffers from long-range vanishing gradients)
Downsampled Bi-LSTM (Small Data)	GloVe 100D	17,568	0.6039	60.8%	Low lexical bias, but severely underfitted due to small data size
Our Attention Bi-LSTM (Full Corpus)	**GloVe 100D Pretrained**	**89,022**	**0.7266**	**72.57%**	**Zero shortcut bias; superior contextual generalization**

The benchmark comparison underscores that combining the 89,022-sample balanced multi-source corpus with pre-trained GloVe embeddings and Bahdanau Additive Attention yields a resilient, high-fidelity classification system that performs reliably across literal praise, situational irony, and caustic sarcasm.